Optimization 2

ISE 5406

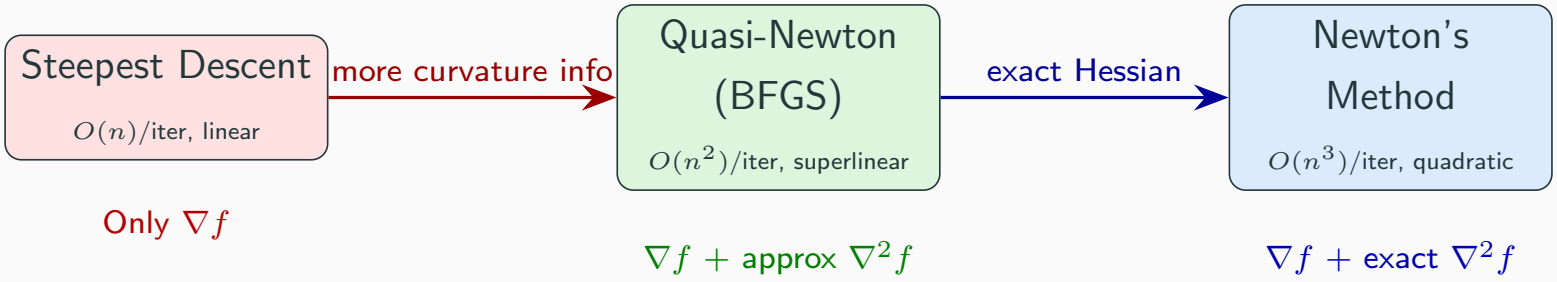
Quasi-Newton Methods: The BFGS Algorithm

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

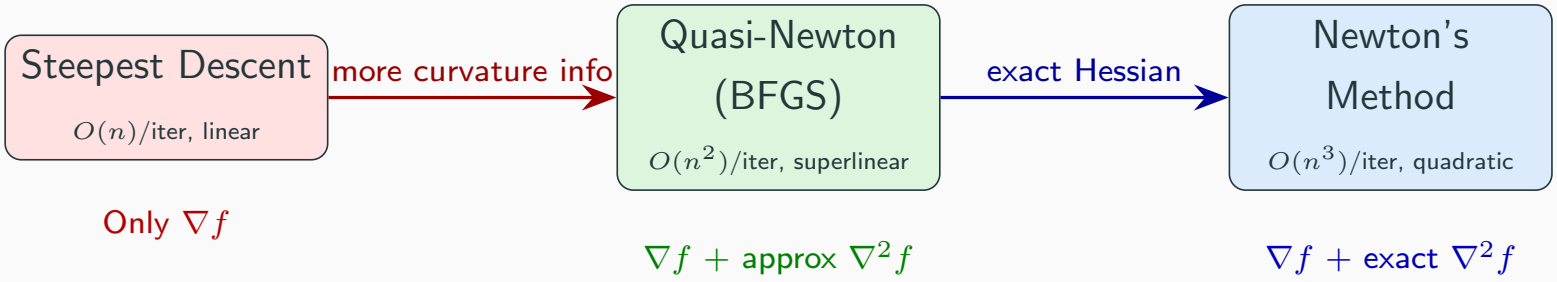
Roadmap

The Optimization Methods Landscape



Key question: Can we get close to Newton's convergence rate while using only gradient information?

The Optimization Methods Landscape

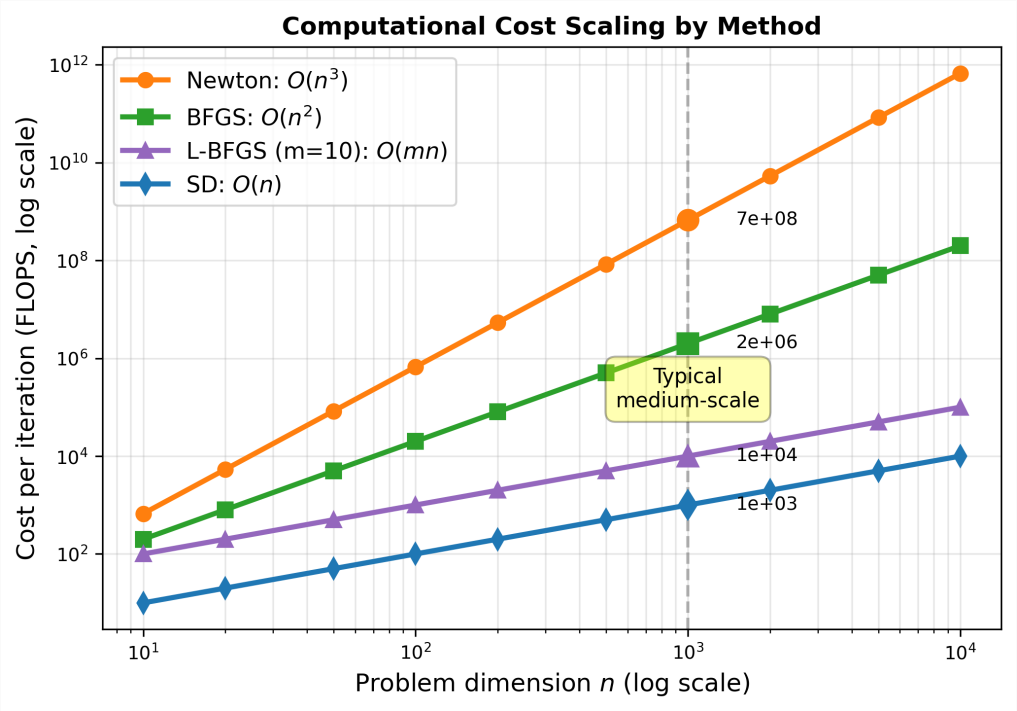


Key question: Can we get close to Newton's convergence rate while using only gradient information?

Answer: Yes! Quasi-Newton methods—especially BFGS—build an approximate Hessian from gradient differences, achieving:

- Superlinear convergence (nearly as fast as quadratic)
- $O(n^2)$ per iteration (no linear system solve if we update the inverse)
- Global convergence with Wolfe line search

Computational Cost: Why It Matters



Takeaway: For large-scale problems ($n > 1000$), the gap between Newton ($O(n^3)$) and BFGS ($O(n^2)$) is enormous. BFGS offers a sweet spot: much faster per iteration than Newton, much better convergence than SD.

Roadmap

Approximating the Hessian from Gradient Information

Fundamental idea: At iteration k, we have seen pairs $(\mathbf{x}_k, \nabla f(\mathbf{x}_k))$.

Can we extract curvature information from gradient *differences*?

Approximating the Hessian from Gradient Information

Fundamental idea: At iteration k, we have seen pairs $(\mathbf{x}_k, \nabla f(\mathbf{x}_k))$.
Can we extract curvature information from gradient *differences*?

By the mean value theorem (multivariate): for smooth f,

$$\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \approx \nabla^2 f(\mathbf{x}_k)(\mathbf{x}_{k+1} - \mathbf{x}_k)$$

Approximating the Hessian from Gradient Information

Fundamental idea: At iteration k, we have seen pairs $(\mathbf{x}_k, \nabla f(\mathbf{x}_k))$.
Can we extract curvature information from gradient *differences*?

By the mean value theorem (multivariate): for smooth f,

$$\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \approx \nabla^2 f(\mathbf{x}_k)(\mathbf{x}_{k+1} - \mathbf{x}_k)$$

Define the key quantities:

$$\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k \qquad \text{(the step)}$$
$$\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \qquad \text{(the gradient difference)}$$

Approximating the Hessian from Gradient Information

Fundamental idea: At iteration k, we have seen pairs $(\mathbf{x}_k, \nabla f(\mathbf{x}_k))$. Can we extract curvature information from gradient *differences*?

By the mean value theorem (multivariate): for smooth f,

$$\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \approx \nabla^2 f(\mathbf{x}_k)(\mathbf{x}_{k+1} - \mathbf{x}_k)$$

Define the key quantities:

$$\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k \qquad \text{(the step)}$$
$$\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k) \qquad \text{(the gradient difference)}$$

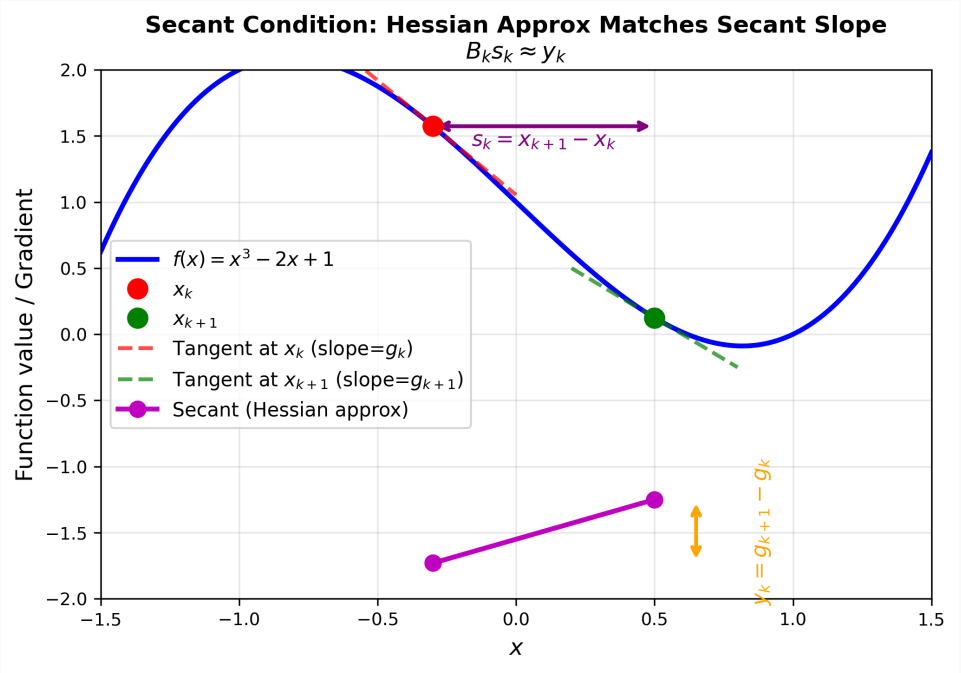
The Secant Condition

Require the new Hessian approximation B_{k+1} to satisfy:

$$\boxed{B_{k+1}\mathbf{s}_k = \mathbf{y}_k}$$

This ensures B_{k+1} matches the actual curvature of f along the most recent step direction.

Geometric Picture of the Secant Condition



In 1D, the secant condition says: the "Hessian approximation" B_{k+1} equals the slope of the secant line through the two most recent gradient evaluations:

$$B_{k+1} = \frac{f'(x_{k+1}) - f'(x_k)}{x_{k+1} - x_k} = \frac{y_k}{s_k}$$

In n dimensions, this becomes the matrix equation $B_{k+1}\mathbf{s}_k = \mathbf{y}_k$.

The Secant Condition Is Underdetermined

Problem: The secant condition $B_{k+1}\mathbf{s}_k = \mathbf{y}_k$ provides only n constraints on B_{k+1}.

But a symmetric $n \times n$ matrix has $\frac{n(n+1)}{2}$ free parameters.

The Secant Condition Is Underdetermined

Problem: The secant condition $B_{k+1}\mathbf{s}_k = \mathbf{y}_k$ provides only n constraints on B_{k+1}.

But a symmetric $n \times n$ matrix has $\frac{n(n+1)}{2}$ free parameters.

Resolution: Impose additional conditions on B_{k+1}:

1. Symmetry: $B_{k+1} = B_{k+1}^T$ (matches Hessian symmetry)
2. Closeness to B_k: The update $\Delta_k = B_{k+1} - B_k$ should have minimal rank

The Secant Condition Is Underdetermined

Problem: The secant condition $B_{k+1}\mathbf{s}_k = \mathbf{y}_k$ provides only n constraints on B_{k+1}.

But a symmetric $n \times n$ matrix has $\frac{n(n+1)}{2}$ free parameters.

Resolution: Impose additional conditions on B_{k+1}:

1. Symmetry: $B_{k+1} = B_{k+1}^T$ (matches Hessian symmetry)
2. Closeness to B_k: The update $\Delta_k = B_{k+1} - B_k$ should have minimal rank

$\text{rank}(\Delta_k) = 1$	$\text{rank}(\Delta_k) = 2$
SR1 update	**BFGS update**

The Secant Condition Is Underdetermined

Problem: The secant condition $B_{k+1}\mathbf{s}_k = \mathbf{y}_k$ provides only n constraints on B_{k+1}.

But a symmetric $n \times n$ matrix has $\frac{n(n+1)}{2}$ free parameters.

Resolution: Impose additional conditions on B_{k+1}:

1. Symmetry: $B_{k+1} = B_{k+1}^T$ (matches Hessian symmetry)
2. Closeness to B_k: The update $\Delta_k = B_{k+1} - B_k$ should have minimal rank

$\text{rank}(\Delta_k) = 1$	$\text{rank}(\Delta_k) = 2$
SR1 update	**BFGS update**

Why not rank 0? That would mean $B_{k+1} = B_k$, violating the secant condition (unless it was already satisfied).

Roadmap

Motivation: Why Quasi-Newton?

The Secant Condition

Derivation of Quasi-Newton Updates

Preserving Positive Definiteness

The BFGS Inverse Hessian Update

Worked Example: One BFGS Iteration

The DFP Update and the Broyden Family

The BFGS Algorithm and Convergence

Limited-Memory BFGS (L-BFGS)

Exercises

Warm-Up: The Symmetric Rank-One (SR1) Update

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{v}\mathbf{v}^T$ for some $\alpha \in \mathbb{R}$, $\mathbf{v} \in \mathbb{R}^n$.

Warm-Up: The Symmetric Rank-One (SR1) Update

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{v}\mathbf{v}^T$ for some $\alpha \in \mathbb{R}$, $\mathbf{v} \in \mathbb{R}^n$.

Step 1: **Apply the secant condition:**

$$(B_k + \alpha \mathbf{v}\mathbf{v}^T)\mathbf{s}_k = \mathbf{y}_k \quad \Longrightarrow \quad \alpha(\mathbf{v}^T\mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k\mathbf{s}_k$$

Warm-Up: The Symmetric Rank-One (SR1) Update

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{v}\mathbf{v}^T$ for some $\alpha \in \mathbb{R}$, $\mathbf{v} \in \mathbb{R}^n$.

Step 1: Apply the secant condition:

$$(B_k + \alpha \mathbf{v}\mathbf{v}^T)\mathbf{s}_k = \mathbf{y}_k \quad \Longrightarrow \quad \alpha(\mathbf{v}^T\mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k\mathbf{s}_k$$

Step 2: The RHS is proportional to $\mathbf{v}$:

$$\mathbf{v} = \beta(\mathbf{y}_k - B_k\mathbf{s}_k) \quad \text{for some } \beta \neq 0$$

Warm-Up: The Symmetric Rank-One (SR1) Update

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{v}\mathbf{v}^T$ for some $\alpha \in \mathbb{R}$, $\mathbf{v} \in \mathbb{R}^n$.

Step 1: Apply the secant condition:

$$(B_k + \alpha \mathbf{v}\mathbf{v}^T)\mathbf{s}_k = \mathbf{y}_k \quad \Longrightarrow \quad \alpha(\mathbf{v}^T\mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k\mathbf{s}_k$$

Step 2: The RHS is proportional to $\mathbf{v}$:

$$\mathbf{v} = \beta(\mathbf{y}_k - B_k\mathbf{s}_k) \quad \text{for some } \beta \neq 0$$

Step 3: Substitute back and solve:

$$\alpha\beta^2[(\mathbf{y}_k - B_k\mathbf{s}_k)^T\mathbf{s}_k] = 1$$

Warm-Up: The Symmetric Rank-One (SR1) Update

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{v}\mathbf{v}^T$ for some $\alpha \in \mathbb{R}$, $\mathbf{v} \in \mathbb{R}^n$.

Step 1: Apply the secant condition:

$$(B_k + \alpha \mathbf{v}\mathbf{v}^T)\mathbf{s}_k = \mathbf{y}_k \quad \Longrightarrow \quad \alpha(\mathbf{v}^T\mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k\mathbf{s}_k$$

Step 2: The RHS is proportional to $\mathbf{v}$:

$$\mathbf{v} = \beta(\mathbf{y}_k - B_k\mathbf{s}_k) \quad \text{for some } \beta \neq 0$$

Step 3: Substitute back and solve:

$$\alpha\beta^2[(\mathbf{y}_k - B_k\mathbf{s}_k)^T\mathbf{s}_k] = 1$$

SR1 formula:

$$\boxed{B_{k+1}^{\text{SR1}} = B_k + \frac{(\mathbf{y}_k - B_k\mathbf{s}_k)(\mathbf{y}_k - B_k\mathbf{s}_k)^T}{(\mathbf{y}_k - B_k\mathbf{s}_k)^T\mathbf{s}_k}}$$

Warm-Up: The Symmetric Rank-One (SR1) Update

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{v}\mathbf{v}^T$ for some $\alpha \in \mathbb{R}$, $\mathbf{v} \in \mathbb{R}^n$.

Step 1: Apply the secant condition:

$$(B_k + \alpha \mathbf{v}\mathbf{v}^T)\mathbf{s}_k = \mathbf{y}_k \quad \Longrightarrow \quad \alpha(\mathbf{v}^T\mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k\mathbf{s}_k$$

Step 2: The RHS is proportional to $\mathbf{v}$:

$$\mathbf{v} = \beta(\mathbf{y}_k - B_k\mathbf{s}_k) \quad \text{for some } \beta \neq 0$$

Step 3: Substitute back and solve:

$$\alpha\beta^2[(\mathbf{y}_k - B_k\mathbf{s}_k)^T\mathbf{s}_k] = 1$$

SR1 formula:

$$\boxed{B_{k+1}^{\text{SR1}} = B_k + \frac{(\mathbf{y}_k - B_k\mathbf{s}_k)(\mathbf{y}_k - B_k\mathbf{s}_k)^T}{(\mathbf{y}_k - B_k\mathbf{s}_k)^T\mathbf{s}_k}}$$

Problem: The SR1 update does **not** preserve positive definiteness! If $B_k \succ 0$, B_{k+1}^{SR1} may be indefinite.

The BFGS Update: Derivation

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{u}\mathbf{u}^T + \beta \mathbf{v}\mathbf{v}^T$ (rank-2 symmetric correction).

The BFGS Update: Derivation

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{u}\mathbf{u}^T + \beta \mathbf{v}\mathbf{v}^T$ (rank-2 symmetric correction).

Step 1: Apply the secant condition:

$$\alpha(\mathbf{u}^T \mathbf{s}_k)\mathbf{u} + \beta(\mathbf{v}^T \mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k \mathbf{s}_k$$

The BFGS Update: Derivation

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{u}\mathbf{u}^T + \beta \mathbf{v}\mathbf{v}^T$ (rank-2 symmetric correction).

Step 1: **Apply the secant condition:**

$$\alpha(\mathbf{u}^T \mathbf{s}_k)\mathbf{u} + \beta(\mathbf{v}^T \mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k \mathbf{s}_k$$

Step 2: **Match the two terms separately** (a natural splitting):

$$\alpha(\mathbf{u}^T \mathbf{s}_k)\mathbf{u} = \mathbf{y}_k, \qquad \beta(\mathbf{v}^T \mathbf{s}_k)\mathbf{v} = -B_k \mathbf{s}_k$$

The BFGS Update: Derivation

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{u}\mathbf{u}^T + \beta \mathbf{v}\mathbf{v}^T$ (rank-2 symmetric correction).

Step 1: **Apply the secant condition:**

$$\alpha(\mathbf{u}^T \mathbf{s}_k)\mathbf{u} + \beta(\mathbf{v}^T \mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k \mathbf{s}_k$$

Step 2: **Match the two terms separately** (a natural splitting):

$$\alpha(\mathbf{u}^T \mathbf{s}_k)\mathbf{u} = \mathbf{y}_k, \qquad \beta(\mathbf{v}^T \mathbf{s}_k)\mathbf{v} = -B_k \mathbf{s}_k$$

Step 3: **From the first:** $\mathbf{u} \| \mathbf{y}_k$, take $\mathbf{u} = \mathbf{y}_k$. Then $\alpha = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$.

Step 4: **From the second:** $\mathbf{v} \| B_k \mathbf{s}_k$, take $\mathbf{v} = B_k \mathbf{s}_k$. Then $\beta = -\frac{1}{\mathbf{s}_k^T B_k \mathbf{s}_k}$.

The BFGS Update: Derivation

Ansatz: $B_{k+1} = B_k + \alpha \mathbf{u}\mathbf{u}^T + \beta \mathbf{v}\mathbf{v}^T$ (rank-2 symmetric correction).

Step 1: Apply the secant condition:

$$\alpha(\mathbf{u}^T \mathbf{s}_k)\mathbf{u} + \beta(\mathbf{v}^T \mathbf{s}_k)\mathbf{v} = \mathbf{y}_k - B_k \mathbf{s}_k$$

Step 2: Match the two terms separately (a natural splitting):

$$\alpha(\mathbf{u}^T \mathbf{s}_k)\mathbf{u} = \mathbf{y}_k, \qquad \beta(\mathbf{v}^T \mathbf{s}_k)\mathbf{v} = -B_k \mathbf{s}_k$$

Step 3: From the first: $\mathbf{u} \| \mathbf{y}_k$, take $\mathbf{u} = \mathbf{y}_k$. Then $\alpha = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$.

Step 4: From the second: $\mathbf{v} \| B_k \mathbf{s}_k$, take $\mathbf{v} = B_k \mathbf{s}_k$. Then $\beta = -\frac{1}{\mathbf{s}_k^T B_k \mathbf{s}_k}$.

BFGS formula (Broyden–Fletcher–Goldfarb–Shanno):

$$\boxed{B_{k+1}^{\text{BFGS}} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}}$$

Understanding the BFGS Update Geometrically

The BFGS update consists of two rank-1 corrections:

B_k

Current approx.

$-$

$$\frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$$

"Forget" direction $\mathbf{s}_k$

$+$

$$\frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$$

"Learn" new curvature

Understanding the BFGS Update Geometrically

The BFGS update consists of two rank-1 corrections:

$$B_k \quad \text{Current approx.} \qquad - \qquad \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k} \quad \text{“Forget” direction } \mathbf{s}_k \qquad + \qquad \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \quad \text{“Learn” new curvature}$$

Interpretation:

1. **Subtraction:** Remove the component of B_k along $\mathbf{s}_k$. This is a rank-1 projection: it “forgets” the old curvature in the step direction.
2. **Addition:** Insert new curvature information from the gradient difference $\mathbf{y}_k$. This is a rank-1 correction: it “learns” the curvature along $\mathbf{s}_k$.

Understanding the BFGS Update Geometrically

The BFGS update consists of two rank-1 corrections:

B_k
Current approx.

$-$

$\frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$
"Forget" direction $\mathbf{s}_k$

$+$

$\frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$
"Learn" new curvature

Interpretation:

1. **Subtraction:** Remove the component of B_k along $\mathbf{s}_k$. This is a rank-1 projection: it "forgets" the old curvature in the step direction.
2. **Addition:** Insert new curvature information from the gradient difference $\mathbf{y}_k$. This is a rank-1 correction: it "learns" the curvature along $\mathbf{s}_k$.

The result: B_{k+1} agrees with B_k in all directions orthogonal to $\mathbf{s}_k$, but matches the true curvature along $\mathbf{s}_k$.

Roadmap

Theorem: BFGS Preserves Positive Definiteness

Theorem (BFGS PD Preservation)
If $B_k \succ 0$ (positive definite) and $\mathbf{y}_k^T \mathbf{s}_k > 0$, then $B_{k+1}^{\text{BFGS}} \succ 0$.

Theorem: BFGS Preserves Positive Definiteness

Theorem (BFGS PD Preservation)
If $B_k \succ 0$ (positive definite) and $\mathbf{y}_k^T \mathbf{s}_k > 0$, then $B_{k+1}^{\text{BFGS}} \succ 0$.

Proof. Let $\mathbf{z} \in \mathbb{R}^n$ be arbitrary with $\mathbf{z} \neq \mathbf{0}$. We must show $\mathbf{z}^T B_{k+1} \mathbf{z} > 0$.

Theorem: BFGS Preserves Positive Definiteness

Theorem (BFGS PD Preservation)
If $B_k \succ 0$ (positive definite) and $\mathbf{y}_k^T \mathbf{s}_k > 0$, then $B_{k+1}^{\text{BFGS}} \succ 0$.

Proof. Let $\mathbf{z} \in \mathbb{R}^n$ be arbitrary with $\mathbf{z} \neq \mathbf{0}$. We must show $\mathbf{z}^T B_{k+1} \mathbf{z} > 0$.

Step 1: Decompose the quadratic form:

$$\mathbf{z}^T B_{k+1} \mathbf{z} = \underbrace{\mathbf{z}^T B_k \mathbf{z} - \frac{(\mathbf{z}^T B_k \mathbf{s}_k)^2}{\mathbf{s}_k^T B_k \mathbf{s}_k}}_{\text{Term } A} + \underbrace{\frac{(\mathbf{z}^T \mathbf{y}_k)^2}{\mathbf{y}_k^T \mathbf{s}_k}}_{\text{Term } B}$$

Theorem: BFGS Preserves Positive Definiteness

Theorem (BFGS PD Preservation)
If $B_k \succ 0$ (positive definite) and $\mathbf{y}_k^T \mathbf{s}_k > 0$, then $B_{k+1}^{\text{BFGS}} \succ 0$.

Proof. Let $\mathbf{z} \in \mathbb{R}^n$ be arbitrary with $\mathbf{z} \neq \mathbf{0}$. We must show $\mathbf{z}^T B_{k+1} \mathbf{z} > 0$.

Step 1: Decompose the quadratic form:

$$\mathbf{z}^T B_{k+1} \mathbf{z} = \underbrace{\mathbf{z}^T B_k \mathbf{z} - \frac{(\mathbf{z}^T B_k \mathbf{s}_k)^2}{\mathbf{s}_k^T B_k \mathbf{s}_k}}_{\text{Term } A} + \underbrace{\frac{(\mathbf{z}^T \mathbf{y}_k)^2}{\mathbf{y}_k^T \mathbf{s}_k}}_{\text{Term } B}$$

Step 2: Term $A \geq 0$ by Cauchy–Schwarz with inner product $\langle \mathbf{u}, \mathbf{v} \rangle_{B_k} = \mathbf{u}^T B_k \mathbf{v}$:

$$(\mathbf{z}^T B_k \mathbf{s}_k)^2 \leq (\mathbf{z}^T B_k \mathbf{z})(\mathbf{s}_k^T B_k \mathbf{s}_k)$$

Equality iff $\mathbf{z} = c\,\mathbf{s}_k$ for some scalar c.

Theorem: BFGS Preserves Positive Definiteness

Theorem (BFGS PD Preservation)
If $B_k \succ 0$ (positive definite) and $\mathbf{y}_k^T \mathbf{s}_k > 0$, then $B_{k+1}^{\text{BFGS}} \succ 0$.

Proof. Let $\mathbf{z} \in \mathbb{R}^n$ be arbitrary with $\mathbf{z} \neq \mathbf{0}$. We must show $\mathbf{z}^T B_{k+1} \mathbf{z} > 0$.

Step 1: Decompose the quadratic form:

$$\mathbf{z}^T B_{k+1} \mathbf{z} = \underbrace{\mathbf{z}^T B_k \mathbf{z} - \frac{(\mathbf{z}^T B_k \mathbf{s}_k)^2}{\mathbf{s}_k^T B_k \mathbf{s}_k}}_{\text{Term } A} + \underbrace{\frac{(\mathbf{z}^T \mathbf{y}_k)^2}{\mathbf{y}_k^T \mathbf{s}_k}}_{\text{Term } B}$$

Step 2: Term $A \geq 0$ by Cauchy–Schwarz with inner product $\langle \mathbf{u}, \mathbf{v} \rangle_{B_k} = \mathbf{u}^T B_k \mathbf{v}$:

$$(\mathbf{z}^T B_k \mathbf{s}_k)^2 \leq (\mathbf{z}^T B_k \mathbf{z})(\mathbf{s}_k^T B_k \mathbf{s}_k)$$

Equality iff $\mathbf{z} = c\,\mathbf{s}_k$ for some scalar c.

Step 3: Term $B \geq 0$ since $\mathbf{y}_k^T \mathbf{s}_k > 0$. Equality iff $\mathbf{z} \perp \mathbf{y}_k$.

Proof (continued): Both Terms Cannot Vanish Simultaneously

Step 4: Suppose both terms vanish. Then:

- Term $A = 0 \implies \mathbf{z} = c\,\mathbf{s}_k$ for some $c \neq 0$
- Term $B = 0 \implies \mathbf{z}^T \mathbf{y}_k = 0$, i.e., $\mathbf{z} \perp \mathbf{y}_k$

Proof (continued): Both Terms Cannot Vanish Simultaneously

Step 4: Suppose both terms vanish. Then:

- Term $A = 0 \implies \mathbf{z} = c\,\mathbf{s}_k$ for some $c \neq 0$
- Term $B = 0 \implies \mathbf{z}^T \mathbf{y}_k = 0$, i.e., $\mathbf{z} \perp \mathbf{y}_k$

Combining: $c\,\mathbf{s}_k^T \mathbf{y}_k = 0$. Since $c \neq 0$, this gives $\mathbf{s}_k^T \mathbf{y}_k = 0$.

Proof (continued): Both Terms Cannot Vanish Simultaneously

Step 4: Suppose both terms vanish. Then:

- Term $A = 0 \implies \mathbf{z} = c\,\mathbf{s}_k$ for some $c \neq 0$
- Term $B = 0 \implies \mathbf{z}^T \mathbf{y}_k = 0$, i.e., $\mathbf{z} \perp \mathbf{y}_k$

Combining: $c\,\mathbf{s}_k^T \mathbf{y}_k = 0$. Since $c \neq 0$, this gives $\mathbf{s}_k^T \mathbf{y}_k = 0$.

But we assumed $\mathbf{y}_k^T \mathbf{s}_k > 0$. Contradiction! $\square$

Proof (continued): Both Terms Cannot Vanish Simultaneously

Step 4: Suppose both terms vanish. Then:

- Term $A = 0 \implies \mathbf{z} = c\,\mathbf{s}_k$ for some $c \neq 0$
- Term $B = 0 \implies \mathbf{z}^T \mathbf{y}_k = 0$, i.e., $\mathbf{z} \perp \mathbf{y}_k$

Combining: $c\,\mathbf{s}_k^T \mathbf{y}_k = 0$. Since $c \neq 0$, this gives $\mathbf{s}_k^T \mathbf{y}_k = 0$.

But we assumed $\mathbf{y}_k^T \mathbf{s}_k > 0$. Contradiction! □

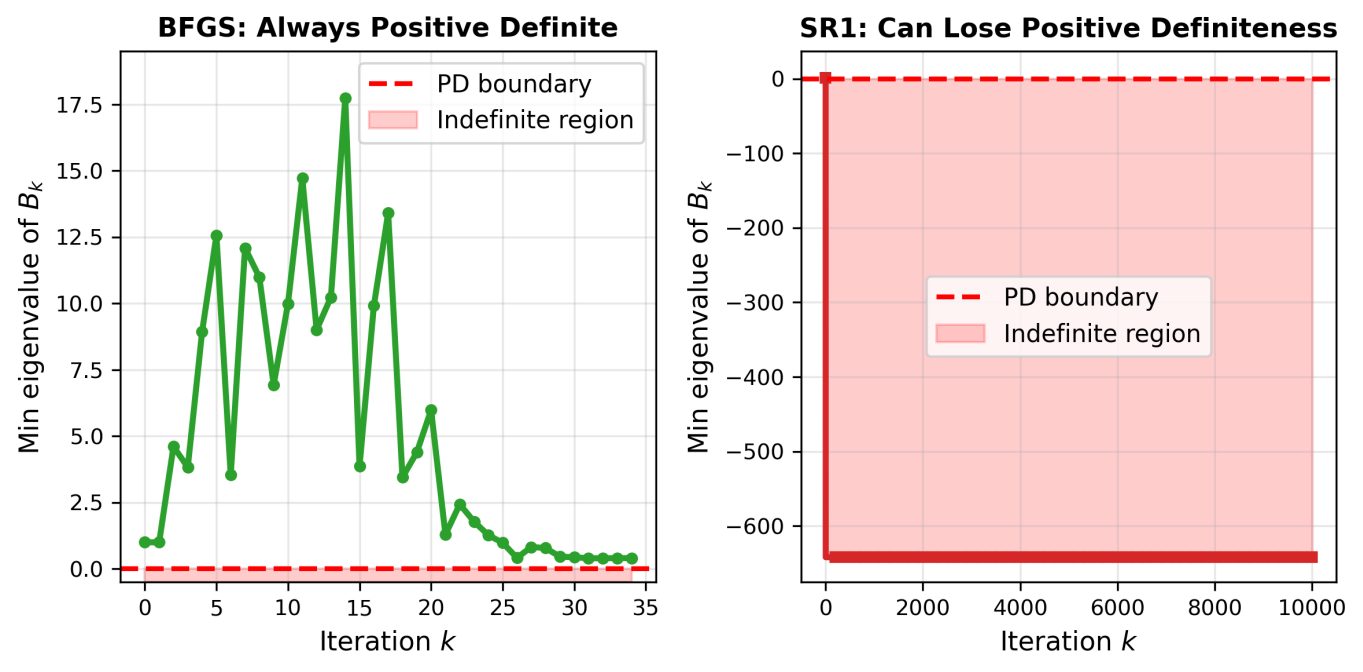
The Critical Condition: $\mathbf{y}_k^T \mathbf{s}_k > 0$

- This condition is called the **curvature condition**
- It is automatically guaranteed by the **Wolfe conditions**:

  The strong Wolfe curvature condition says
  $|\nabla f(\mathbf{x}_{k+1})^T \mathbf{d}_k| \leq c_2 |\nabla f(\mathbf{x}_k)^T \mathbf{d}_k|$ for $c_2 < 1$.

  This implies $\mathbf{y}_k^T \mathbf{s}_k = [\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)]^T \mathbf{s}_k > 0$.
- **Practical rule:** Always use a Wolfe line search with BFGS!

SR1 vs. BFGS: Positive Definiteness in Practice



Left: BFGS always maintains $\lambda_{\min}(B_k) > 0$ (positive definite).

Right: SR1 can produce indefinite updates—the minimum eigenvalue drops below zero. This means the SR1 direction may **not** be a descent direction!

Roadmap

Why Update the Inverse Directly?

Recall: The search direction is $\mathbf{d}_k = -B_k^{-1}\nabla f(\mathbf{x}_k)$.

Two implementation strategies:

Strategy 1: Store B_k

Update $B_k \to B_{k+1}$ via BFGS

Solve $B_{k+1}\mathbf{d}_{k+1} = -\nabla f(\mathbf{x}_{k+1})$

Cost: $O(n^2)$ update + $O(n^3)$ solve

Strategy 2: Store $H_k = B_k^{-1}$

Update $H_k \to H_{k+1}$ directly

Compute $\mathbf{d}_{k+1} = -H_{k+1}\nabla f(\mathbf{x}_{k+1}$

Cost: $O(n^2)$ update + $O(n^2)$ mult

Why Update the Inverse Directly?

Recall: The search direction is $\mathbf{d}_k = -B_k^{-1}\nabla f(\mathbf{x}_k)$.

Two implementation strategies:

Strategy 1: Store B_k

Update $B_k \to B_{k+1}$ via BFGS

Solve $B_{k+1}\mathbf{d}_{k+1} = -\nabla f(\mathbf{x}_{k+1})$

Cost: $O(n^2)$ update + $O(n^3)$ solve

Strategy 2: Store $H_k = B_k^{-1}$

Update $H_k \to H_{k+1}$ directly

Compute $\mathbf{d}_{k+1} = -H_{k+1}\nabla f(\mathbf{x}_{k+1}$

Cost: $O(n^2)$ update + $O(n^2)$ mult

Strategy 2 is strictly cheaper! The question is: can we derive an update for $H_k = B_k^{-1}$ directly?

Why Update the Inverse Directly?

Recall: The search direction is $\mathbf{d}_k = -B_k^{-1}\nabla f(\mathbf{x}_k)$.

Two implementation strategies:

Strategy 1: Store B_k
Update $B_k \to B_{k+1}$ via BFGS
Solve $B_{k+1}\mathbf{d}_{k+1} = -\nabla f(\mathbf{x}_{k+1})$
Cost: $O(n^2)$ update + $O(n^3)$ solve

Strategy 2: Store $H_k = B_k^{-1}$
Update $H_k \to H_{k+1}$ directly
Compute $\mathbf{d}_{k+1} = -H_{k+1}\nabla f(\mathbf{x}_{k+1}$
Cost: $O(n^2)$ update + $O(n^2)$ mult

Strategy 2 is strictly cheaper! The question is: can we derive an update for $H_k = B_k^{-1}$ directly?

Yes—using the Sherman–Morrison–Woodbury formula.

Tool: The Sherman–Morrison Formula

Theorem (Sherman–Morrison)
If $A \in \mathbb{R}^{n\times n}$ is invertible and $1 + \mathbf{v}^T A^{-1}\mathbf{u} \neq 0$, then:

$$(A + \mathbf{u}\mathbf{v}^T)^{-1} = A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}$$

Tool: The Sherman–Morrison Formula

Theorem (Sherman–Morrison)
If $A \in \mathbb{R}^{n\times n}$ is invertible and $1 + \mathbf{v}^T A^{-1}\mathbf{u} \neq 0$, then:

$$(A + \mathbf{u}\mathbf{v}^T)^{-1} = A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}$$

Why is this powerful?

- Adding a rank-1 correction to A only requires a rank-1 correction to A^{-1}
- Cost: $O(n^2)$ (two matrix-vector products plus an outer product)
- For rank-2 corrections: apply Sherman–Morrison twice

Tool: The Sherman–Morrison Formula

Theorem (Sherman–Morrison)
If $A \in \mathbb{R}^{n\times n}$ is invertible and $1 + \mathbf{v}^T A^{-1}\mathbf{u} \neq 0$, then:

$$(A + \mathbf{u}\mathbf{v}^T)^{-1} = A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}$$

Why is this powerful?

- Adding a rank-1 correction to A only requires a rank-1 correction to A^{-1}
- Cost: $O(n^2)$ (two matrix-vector products plus an outer product)
- For rank-2 corrections: apply Sherman–Morrison twice

Proof sketch: Multiply $(A + \mathbf{u}\mathbf{v}^T)$ by the proposed inverse and verify the result is I:

$$(A+\mathbf{u}\mathbf{v}^T)\left(A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}\right) = I+\mathbf{u}\mathbf{v}^T A^{-1}-\frac{\mathbf{u}\mathbf{v}^T A^{-1} + \mathbf{u}(\mathbf{v}^T A^{-1}\mathbf{u})\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}} = I. \quad \checkmark$$

Deriving the BFGS Inverse Update

Goal: Given $B_{k+1} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$, find $H_{k+1} = B_{k+1}^{-1}$.

Deriving the BFGS Inverse Update

Goal: Given $B_{k+1} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$, find $H_{k+1} = B_{k+1}^{-1}$.

Step 1: Apply Sherman–Morrison to the positive term:

Let $\tilde{B} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$, so $\mathbf{u} = \frac{\mathbf{y}_k}{\mathbf{y}_k^T \mathbf{s}_k}$, $\mathbf{v} = \mathbf{y}_k$:

$$\tilde{B}^{-1} = H_k - \frac{H_k \mathbf{y}_k \mathbf{y}_k^T H_k}{\mathbf{y}_k^T \mathbf{s}_k + \mathbf{y}_k^T H_k \mathbf{y}_k}$$

Deriving the BFGS Inverse Update

Goal: Given $B_{k+1} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$, find $H_{k+1} = B_{k+1}^{-1}$.

Step 1: Apply Sherman–Morrison to the positive term:

Let $\tilde{B} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$, so $\mathbf{u} = \frac{\mathbf{y}_k}{\mathbf{y}_k^T \mathbf{s}_k}$, $\mathbf{v} = \mathbf{y}_k$:

$$\tilde{B}^{-1} = H_k - \frac{H_k \mathbf{y}_k \mathbf{y}_k^T H_k}{\mathbf{y}_k^T \mathbf{s}_k + \mathbf{y}_k^T H_k \mathbf{y}_k}$$

Step 2: Apply Sherman–Morrison again for the negative term.

$B_{k+1} = \tilde{B} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$: another rank-1 update to $\tilde{B}$.

Deriving the BFGS Inverse Update

Goal: Given $B_{k+1} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$, find $H_{k+1} = B_{k+1}^{-1}$.

Step 1: Apply Sherman–Morrison to the positive term:

Let $\tilde{B} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$, so $\mathbf{u} = \frac{\mathbf{y}_k}{\mathbf{y}_k^T \mathbf{s}_k}$, $\mathbf{v} = \mathbf{y}_k$:

$$\tilde{B}^{-1} = H_k - \frac{H_k \mathbf{y}_k \mathbf{y}_k^T H_k}{\mathbf{y}_k^T \mathbf{s}_k + \mathbf{y}_k^T H_k \mathbf{y}_k}$$

Step 2: Apply Sherman–Morrison again for the negative term.

$B_{k+1} = \tilde{B} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$: another rank-1 update to $\tilde{B}$.

Step 3: After careful algebra (see Nocedal & Wright, §6.1), the result simplifies beautifully.

Let $\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$:

$$\boxed{H_{k+1} = (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T)\, H_k\, (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T}$$

The BFGS Inverse Update: Compact Form

Denoting $V_k = I - \rho_k \mathbf{y}_k \mathbf{s}_k^T$ where $\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$:

$$\boxed{H_{k+1} = V_k^T \, H_k \, V_k + \rho_k \, \mathbf{s}_k \mathbf{s}_k^T}$$

The BFGS Inverse Update: Compact Form

Denoting $V_k = I - \rho_k \mathbf{y}_k \mathbf{s}_k^T$ where $\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$:

$$\boxed{H_{k+1} = V_k^T \, H_k \, V_k + \rho_k \, \mathbf{s}_k \mathbf{s}_k^T}$$

Properties:

1. Symmetric: $H_{k+1} = H_{k+1}^T$ (by construction)
2. Positive definite: if $H_k \succ 0$ and $\mathbf{y}_k^T \mathbf{s}_k > 0$
3. Inverse secant condition: $H_{k+1} \mathbf{y}_k = \mathbf{s}_k$ (verify by direct computation)
4. Cost: $O(n^2)$ (one matrix–matrix product, one outer product)

The BFGS Inverse Update: Compact Form

Denoting $V_k = I - \rho_k \mathbf{y}_k \mathbf{s}_k^T$ where $\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$:

$$\boxed{H_{k+1} = V_k^T \, H_k \, V_k + \rho_k \, \mathbf{s}_k \mathbf{s}_k^T}$$

Properties:

1. Symmetric: $H_{k+1} = H_{k+1}^T$ (by construction)
2. Positive definite: if $H_k \succ 0$ and $\mathbf{y}_k^T \mathbf{s}_k > 0$
3. Inverse secant condition: $H_{k+1} \mathbf{y}_k = \mathbf{s}_k$ (verify by direct computation)
4. Cost: $O(n^2)$ (one matrix–matrix product, one outer product)

Computing the search direction—just a matrix-vector product:

$$\mathbf{d}_{k+1} = -H_{k+1} \nabla f(\mathbf{x}_{k+1}) \qquad \text{cost: } O(n^2)$$

Compare: Newton requires solving $\nabla^2 f(\mathbf{x}_k) \mathbf{d}_k = -\nabla f(\mathbf{x}_k)$ at cost $O(n^3)$.

Roadmap

Example: Forward BFGS Update

Data: $B_1 = I_2, \quad \mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{y}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$

Example: Forward BFGS Update

Data: $B_1 = I_2, \quad \mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{y}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$

Step 1: **Verify curvature condition:** $\mathbf{y}_1^T \mathbf{s}_1 = 2 > 0.$ ✓

Example: Forward BFGS Update

Data: $B_1 = I_2, \quad \mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{y}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$

Step 1: **Verify curvature condition:** $\mathbf{y}_1^T \mathbf{s}_1 = 2 > 0.$ ✓

Step 2: **Compute intermediate quantities:**

$$B_1 \mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \mathbf{s}_1^T B_1 \mathbf{s}_1 = 1$$

$$\frac{B_1 \mathbf{s}_1 \mathbf{s}_1^T B_1}{\mathbf{s}_1^T B_1 \mathbf{s}_1} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad \frac{\mathbf{y}_1 \mathbf{y}_1^T}{\mathbf{y}_1^T \mathbf{s}_1} = \frac{1}{2} \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & \frac{1}{2} \end{pmatrix}$$

Example: Forward BFGS Update

Data: $B_1 = I_2, \quad \mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{y}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$

Step 1: Verify curvature condition: $\mathbf{y}_1^T \mathbf{s}_1 = 2 > 0.$ ✓

Step 2: Compute intermediate quantities:

$$B_1 \mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \mathbf{s}_1^T B_1 \mathbf{s}_1 = 1$$

$$\frac{B_1 \mathbf{s}_1 \mathbf{s}_1^T B_1}{\mathbf{s}_1^T B_1 \mathbf{s}_1} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad \frac{\mathbf{y}_1 \mathbf{y}_1^T}{\mathbf{y}_1^T \mathbf{s}_1} = \frac{1}{2} \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & \frac{1}{2} \end{pmatrix}$$

Step 3: Assemble:

$$B_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 2 & 1 \\ 1 & \frac{1}{2} \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & \frac{3}{2} \end{pmatrix}$$

Example: Forward BFGS Update

Data: $B_1 = I_2, \quad \mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{y}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$

Step 1: Verify curvature condition: $\mathbf{y}_1^T \mathbf{s}_1 = 2 > 0.$ ✓

Step 2: Compute intermediate quantities:

$$B_1 \mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \mathbf{s}_1^T B_1 \mathbf{s}_1 = 1$$

$$\frac{B_1 \mathbf{s}_1 \mathbf{s}_1^T B_1}{\mathbf{s}_1^T B_1 \mathbf{s}_1} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad \frac{\mathbf{y}_1 \mathbf{y}_1^T}{\mathbf{y}_1^T \mathbf{s}_1} = \frac{1}{2} \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & \frac{1}{2} \end{pmatrix}$$

Step 3: Assemble:

$$B_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 2 & 1 \\ 1 & \frac{1}{2} \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & \frac{3}{2} \end{pmatrix}$$

Step 4: Verify: $B_2 \mathbf{s}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \mathbf{y}_1.$ ✓ $\quad \det(B_2) = 3 - 1 = 2 > 0,$

eigenvalues $\lambda \approx 2.78, 0.72.$ ✓

Example: Inverse BFGS Update

Same data: $H_1 = B_1^{-1} = I_2$, $\mathbf{s}_1 = (1, 0)^T$, $\mathbf{y}_1 = (2, 1)^T$, $\rho_1 = \frac{1}{2}$.

Example: Inverse BFGS Update

Same data: $H_1 = B_1^{-1} = I_2$, $\mathbf{s}_1 = (1,0)^T$, $\mathbf{y}_1 = (2,1)^T$, $\rho_1 = \frac{1}{2}$.

Step 1: Compute auxiliary matrices:

$$V_1 = I - \rho_1 \mathbf{y}_1 \mathbf{s}_1^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{2}\begin{pmatrix} 2 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ -\frac{1}{2} & 1 \end{pmatrix}$$

$$W_1 = I - \rho_1 \mathbf{s}_1 \mathbf{y}_1^T = \begin{pmatrix} 0 & -\frac{1}{2} \\ 0 & 1 \end{pmatrix}$$

Example: Inverse BFGS Update

Same data: $H_1 = B_1^{-1} = I_2$, $\mathbf{s}_1 = (1,0)^T$, $\mathbf{y}_1 = (2,1)^T$, $\rho_1 = \frac{1}{2}$.

Step 1: Compute auxiliary matrices:

$$V_1 = I - \rho_1 \mathbf{y}_1 \mathbf{s}_1^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{2}\begin{pmatrix} 2 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ -\frac{1}{2} & 1 \end{pmatrix}$$

$$W_1 = I - \rho_1 \mathbf{s}_1 \mathbf{y}_1^T = \begin{pmatrix} 0 & -\frac{1}{2} \\ 0 & 1 \end{pmatrix}$$

Step 2: Compute H_2:

$$H_2 = W_1 H_1 V_1 + \rho_1 \mathbf{s}_1 \mathbf{s}_1^T = \begin{pmatrix} \frac{1}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix} + \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} \frac{3}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix}$$

Example: Inverse BFGS Update

Same data: $H_1 = B_1^{-1} = I_2$, $\mathbf{s}_1 = (1,0)^T$, $\mathbf{y}_1 = (2,1)^T$, $\rho_1 = \frac{1}{2}$.

Step 1: Compute auxiliary matrices:

$$V_1 = I - \rho_1 \mathbf{y}_1 \mathbf{s}_1^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{2}\begin{pmatrix} 2 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ -\frac{1}{2} & 1 \end{pmatrix}$$

$$W_1 = I - \rho_1 \mathbf{s}_1 \mathbf{y}_1^T = \begin{pmatrix} 0 & -\frac{1}{2} \\ 0 & 1 \end{pmatrix}$$

Step 2: Compute H_2:

$$H_2 = W_1 H_1 V_1 + \rho_1 \mathbf{s}_1 \mathbf{s}_1^T = \begin{pmatrix} \frac{1}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix} + \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} \frac{3}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix}$$

Step 3: Verify inverse secant condition:

$$H_2 \mathbf{y}_1 = \begin{pmatrix} \frac{3}{2} - \frac{1}{2} \\ -1 + 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \mathbf{s}_1. \ \checkmark$$

Example: Inverse BFGS Update

Same data: $H_1 = B_1^{-1} = I_2$, $\mathbf{s}_1 = (1,0)^T$, $\mathbf{y}_1 = (2,1)^T$, $\rho_1 = \frac{1}{2}$.

Step 1: Compute auxiliary matrices:

$$V_1 = I - \rho_1 \mathbf{y}_1 \mathbf{s}_1^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{2}\begin{pmatrix} 2 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ -\frac{1}{2} & 1 \end{pmatrix}$$

$$W_1 = I - \rho_1 \mathbf{s}_1 \mathbf{y}_1^T = \begin{pmatrix} 0 & -\frac{1}{2} \\ 0 & 1 \end{pmatrix}$$

Step 2: Compute H_2:

$$H_2 = W_1 H_1 V_1 + \rho_1 \mathbf{s}_1 \mathbf{s}_1^T = \begin{pmatrix} \frac{1}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix} + \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} \frac{3}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix}$$

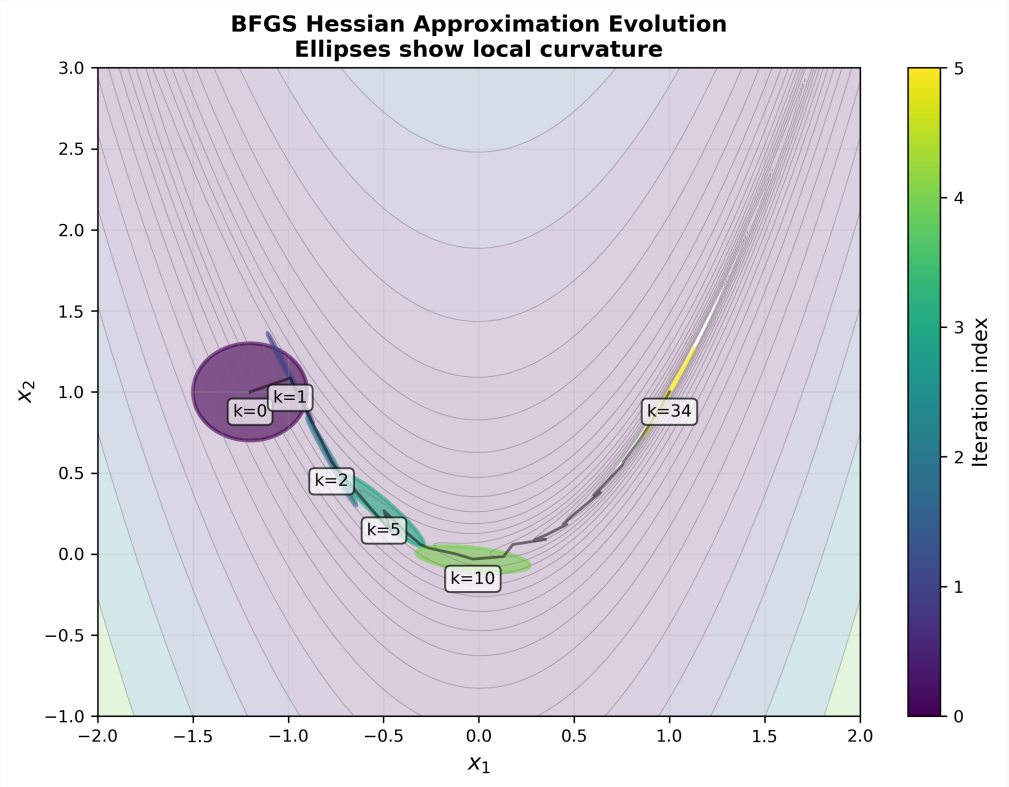
Step 3: Verify inverse secant condition:

$$H_2 \mathbf{y}_1 = \begin{pmatrix} \frac{3}{2} - \frac{1}{2} \\ -1 + 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \mathbf{s}_1. \ \checkmark$$

Step 4: Verify $H_2 = B_2^{-1}$: $\det(B_2) = 2$, so

$$B_2^{-1} = \frac{1}{2}\begin{pmatrix} \frac{3}{2} & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} \frac{3}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix} = H_2. \ \checkmark$$

How the Hessian Approximation Evolves



Each ellipse shows the level set $\{\mathbf{z} : \mathbf{z}^T B_k \mathbf{z} = 1\}$ at the corresponding iterate. Initially $B_0 = I$ (circle). Over iterations, the BFGS approximation deforms to match the true local curvature, becoming more elongated in the steep direction and flatter in the gentle direction.

Roadmap

The DFP Update

Historical note: The DFP method (Davidon 1959, Fletcher–Powell 1963) was the first quasi-Newton method.

Idea: Apply the same rank-2 derivation directly to $H_k = B_k^{-1}$, requiring the inverse secant condition $H_{k+1}\mathbf{y}_k = \mathbf{s}_k$:

$$\boxed{H_{k+1}^{\text{DFP}} = H_k + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{H_k \mathbf{y}_k \mathbf{y}_k^T H_k}{\mathbf{y}_k^T H_k \mathbf{y}_k}}$$

The DFP Update

Historical note: The DFP method (Davidon 1959, Fletcher–Powell 1963) was the first quasi-Newton method.

Idea: Apply the same rank-2 derivation directly to $H_k = B_k^{-1}$, requiring the inverse secant condition $H_{k+1}\mathbf{y}_k = \mathbf{s}_k$:

$$\boxed{H_{k+1}^{\text{DFP}} = H_k + \frac{\mathbf{s}_k\mathbf{s}_k^T}{\mathbf{y}_k^T\mathbf{s}_k} - \frac{H_k\mathbf{y}_k\mathbf{y}_k^T H_k}{\mathbf{y}_k^T H_k\mathbf{y}_k}}$$

Duality between BFGS and DFP:

	Updates B_k	**Updates $H_k = B_k^{-1}$**
BFGS	$B_k + \frac{\mathbf{y}_k\mathbf{y}_k^T}{\mathbf{y}_k^T\mathbf{s}_k} - \frac{B_k\mathbf{s}_k\mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k\mathbf{s}_k}$	via Sherman–Morrison
DFP	via Sherman–Morrison	$H_k + \frac{\mathbf{s}_k\mathbf{s}_k^T}{\mathbf{y}_k^T\mathbf{s}_k} - \frac{H_k\mathbf{y}_k\mathbf{y}_k^T H_k}{\mathbf{y}_k^T H_k\mathbf{y}_k}$

The DFP Update

Historical note: The DFP method (Davidon 1959, Fletcher–Powell 1963) was the first quasi-Newton method.

Idea: Apply the same rank-2 derivation directly to $H_k = B_k^{-1}$, requiring the inverse secant condition $H_{k+1}\mathbf{y}_k = \mathbf{s}_k$:

$$\boxed{H_{k+1}^{\text{DFP}} = H_k + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{H_k \mathbf{y}_k \mathbf{y}_k^T H_k}{\mathbf{y}_k^T H_k \mathbf{y}_k}}$$

Duality between BFGS and DFP:

	Updates B_k	**Updates $H_k = B_k^{-1}$**
BFGS	$B_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^T B_k}{\mathbf{s}_k^T B_k \mathbf{s}_k}$	via Sherman–Morrison
DFP	via Sherman–Morrison	$H_k + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{H_k \mathbf{y}_k \mathbf{y}_k^T H_k}{\mathbf{y}_k^T H_k \mathbf{y}_k}$

Observe: the formulas are the same structure with $\mathbf{s}_k \leftrightarrow \mathbf{y}_k$ and $B_k \leftrightarrow H_k$ swapped!

The Broyden Family

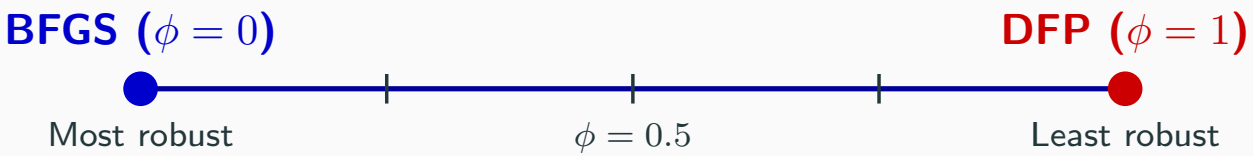
Both BFGS and DFP are special cases of a one-parameter family:

$$B_{k+1}^{(\phi)} = (1-\phi)\, B_{k+1}^{\mathsf{BFGS}} + \phi\, B_{k+1}^{\mathsf{DFP}}, \quad \phi \in [0,1]$$

The Broyden Family

Both BFGS and DFP are special cases of a one-parameter family:

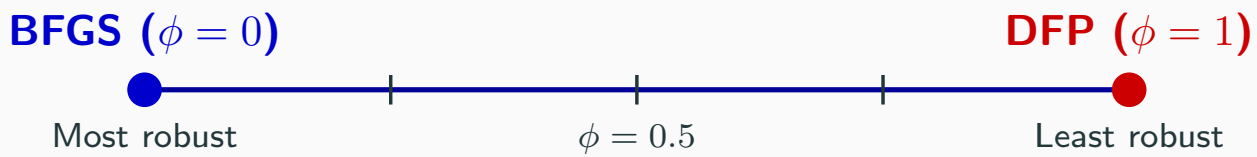
$$B_{k+1}^{(\phi)} = (1-\phi)\, B_{k+1}^{\text{BFGS}} + \phi\, B_{k+1}^{\text{DFP}}, \quad \phi \in [0,1]$$



The Broyden Family

Both BFGS and DFP are special cases of a one-parameter family:

$$B_{k+1}^{(\phi)} = (1-\phi)\, B_{k+1}^{\text{BFGS}} + \phi\, B_{k+1}^{\text{DFP}}, \quad \phi \in [0,1]$$



Why BFGS wins in practice:

- BFGS is more robust to inexact line searches
- DFP is more sensitive to $\mathbf{y}_k^T \mathbf{s}_k$ being small
- Empirically, BFGS requires fewer iterations on most problems
- Recommendation: Use BFGS ($\phi = 0$) unless you have a specific reason not to

Roadmap

The Complete BFGS Algorithm

Algorithm: BFGS with Wolfe Line Search
Input: Starting point $\mathbf{x}_1 \in \mathbb{R}^n$, tolerance $\epsilon > 0$.

Initialize: $H_1 = I_n$ (identity matrix), $k = 1$.

1. **Gradient check:** If $\|\nabla f(\mathbf{x}_k)\| < \epsilon$, **stop**: $\mathbf{x}_k$ is the approximate minimizer.
2. **Search direction:** $\mathbf{d}_k = -H_k \nabla f(\mathbf{x}_k)$. [$O(n^2)$: matrix-vector product]
3. **Line search:** Find λ_k satisfying the Wolfe conditions. [backtracking]
4. **Update iterate:** $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{d}_k$.
5. **Compute differences:**

$$\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k = \lambda_k \mathbf{d}_k$$
$$\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$$

6. **BFGS inverse update:**

$$\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}, \quad H_{k+1} = (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) H_k (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T$$

7. Set $k \leftarrow k + 1$, go to Step 1.

Convergence Theory

Theorem (Global Convergence of BFGS)
Let $f : \mathbb{R}^n \to \mathbb{R}$ be twice continuously differentiable, bounded below, with Lipschitz continuous gradient. If the Wolfe conditions are satisfied at each step, then:

$$\lim_{k \to \infty} \nabla f(\mathbf{x}_k) = \mathbf{0}$$

Convergence Theory

Theorem (Global Convergence of BFGS)
Let $f : \mathbb{R}^n \to \mathbb{R}$ be twice continuously differentiable, bounded below, with Lipschitz continuous gradient. If the Wolfe conditions are satisfied at each step, then:

$$\lim_{k \to \infty} \nabla f(\mathbf{x}_k) = \mathbf{0}$$

Theorem (Superlinear Convergence of BFGS (Dennis & Moré, 1974)**)**
Under additional assumptions (f strongly convex near $\mathbf{x}^*$, $\nabla^2 f$ Lipschitz continuous), BFGS converges **superlinearly***:*

$$\lim_{k \to \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|} = 0$$

Convergence Theory

Theorem (Global Convergence of BFGS)
Let $f : \mathbb{R}^n \to \mathbb{R}$ be twice continuously differentiable, bounded below, with Lipschitz continuous gradient. If the Wolfe conditions are satisfied at each step, then:

$$\lim_{k\to\infty} \nabla f(\mathbf{x}_k) = \mathbf{0}$$

Theorem (Superlinear Convergence of BFGS (Dennis & Moré, 1974)**)**
Under additional assumptions (f strongly convex near $\mathbf{x}^*$, $\nabla^2 f$ Lipschitz continuous), BFGS converges **superlinearly***:*

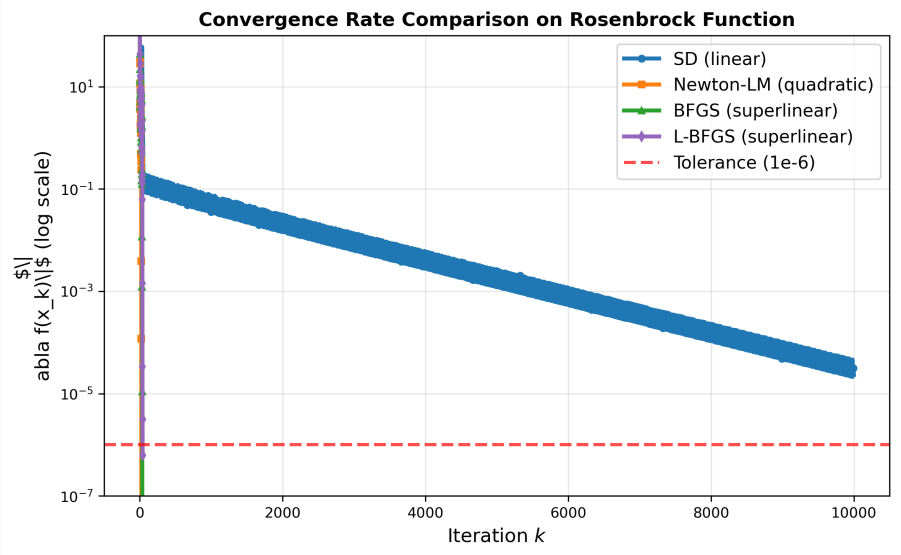
$$\lim_{k\to\infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|} = 0$$

Key insight: Superlinear convergence means BFGS "learns" the Hessian asymptotically:

$$\lim_{k\to\infty} \frac{\|(B_k - \nabla^2 f(\mathbf{x}^*))\mathbf{d}_k\|}{\|\mathbf{d}_k\|} = 0$$

The Hessian approximation becomes exact along the search directions—it does not need to converge to $\nabla^2 f(\mathbf{x}^*)$ globally!

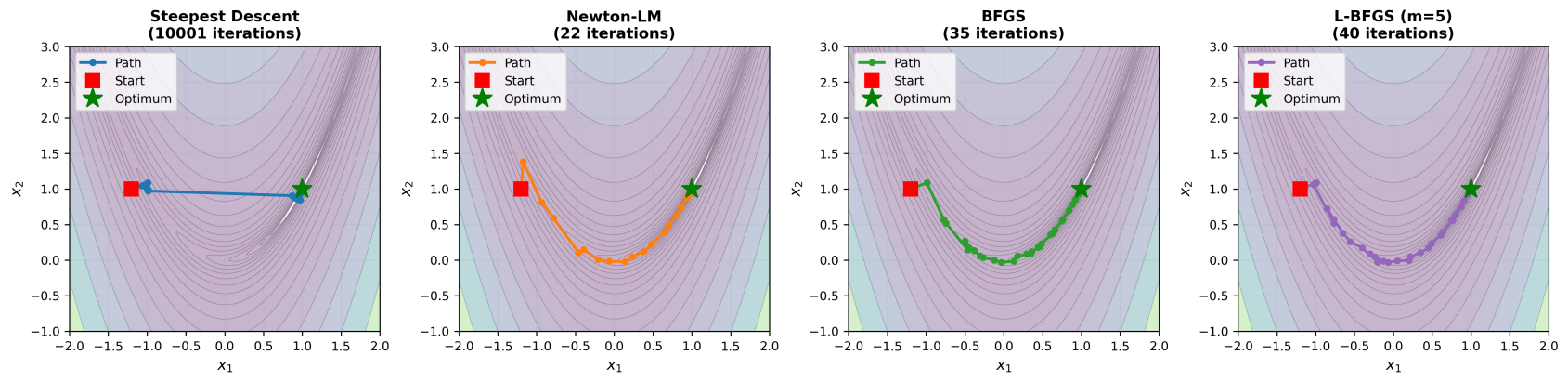
Convergence in Practice: Rosenbrock Function



Observations:

- Steepest descent: slow, linear convergence; gets trapped in the banana valley
- Newton (LM): fast quadratic convergence once near the minimizer
- BFGS: superlinear convergence, nearly as fast as Newton
- L-BFGS: slightly slower but uses $O(n)$ memory instead of $O(n^2)$

BFGS vs. Other Methods on the Rosenbrock Function



$f(\mathbf{x}) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$ starting from $\mathbf{x}_0 = (-1.2, 1)^T$.

Key: BFGS finds the minimizer $\mathbf{x}^* = (1, 1)^T$ efficiently—avoiding the zigzag of SD and the Hessian cost of Newton.

Roadmap

The Memory Problem

Issue: Storing $H_k \in \mathbb{R}^{n \times n}$ requires $O(n^2)$ memory.

For $n = 10^6$ (typical in machine learning): H_k would need ~ 8 TB of memory!

The Memory Problem

Issue: Storing $H_k \in \mathbb{R}^{n \times n}$ requires $O(n^2)$ memory.

For $n = 10^6$ (typical in machine learning): H_k would need ~ 8 TB of memory!

L-BFGS idea: Don't store H_k explicitly. Instead, store only the last m pairs $\{(\mathbf{s}_i, \mathbf{y}_i)\}_{i=k-m}^{k-1}$ and implicitly compute $H_k \nabla f(\mathbf{x}_k)$.

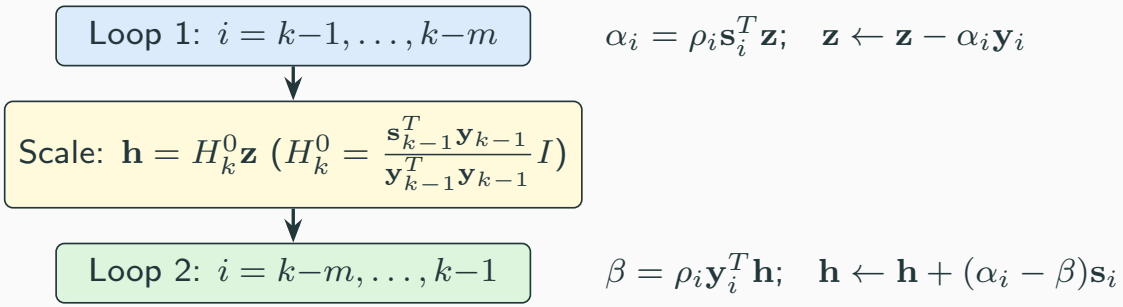
The Memory Problem

Issue: Storing $H_k \in \mathbb{R}^{n \times n}$ requires $O(n^2)$ memory.

For $n = 10^6$ (typical in machine learning): H_k would need ~ 8 TB of memory!

L-BFGS idea: Don't store H_k explicitly. Instead, store only the last m pairs $\{(\mathbf{s}_i, \mathbf{y}_i)\}_{i=k-m}^{k-1}$ and implicitly compute $H_k \nabla f(\mathbf{x}_k)$.

The two-loop recursion (Nocedal, 1980):



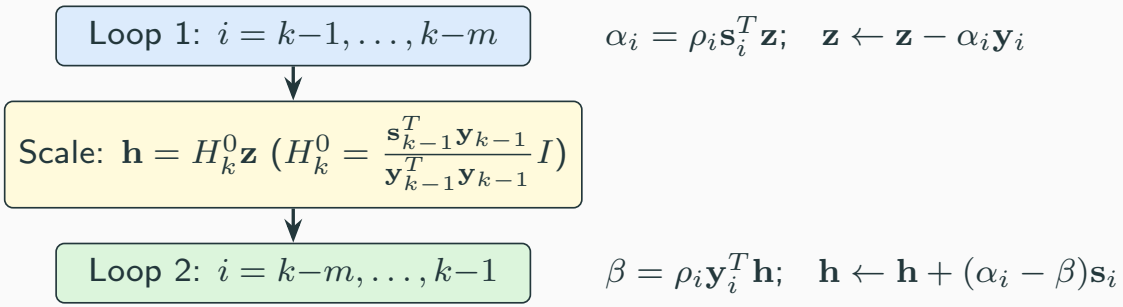
The Memory Problem

Issue: Storing $H_k \in \mathbb{R}^{n \times n}$ requires $O(n^2)$ memory.

For $n = 10^6$ (typical in machine learning): H_k would need ~ 8 TB of memory!

L-BFGS idea: Don't store H_k explicitly. Instead, store only the last m pairs $\{(\mathbf{s}_i, \mathbf{y}_i)\}_{i=k-m}^{k-1}$ and implicitly compute $H_k \nabla f(\mathbf{x}_k)$.

The two-loop recursion (Nocedal, 1980):



Memory: $O(mn)$ (typically $m = 3$–20).

Cost per iteration: $O(mn)$ (vs. $O(n^2)$ for full BFGS).

Roadmap

Exercise 1: BFGS Update Computation

Exercise 1

Consider $f(\mathbf{x}) = x_1^2 + 4x_2^2$. Starting from $\mathbf{x}_1 = (2, 1)^T$ with $B_1 = I_2$.

1. Compute $\nabla f(\mathbf{x}_1)$. Take a steepest descent step with exact line search to get $\mathbf{x}_2$.
2. Compute $\mathbf{s}_1$, $\mathbf{y}_1$, and verify $\mathbf{y}_1^T \mathbf{s}_1 > 0$.
3. Apply the BFGS update to compute B_2. Verify the secant condition $B_2 \mathbf{s}_1 = \mathbf{y}_1$.
4. Compare B_2 with the true Hessian $\nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 8 \end{pmatrix}$. How close is the BFGS approximation after one step?

Solution: Exercise 1

Solution

(1) $\nabla f(\mathbf{x}_1) = (2 \cdot 2,\ 8 \cdot 1)^T = (4, 8)^T$. Direction: $\mathbf{d}_1 = -(4, 8)^T$.

Exact line search: $\lambda_1 = \frac{\nabla f^T \nabla f}{\nabla f^T Q \mathbf{d}_1}$ where $Q = \text{diag}(2, 8)$.

$\lambda_1 = \frac{16+64}{2 \cdot 16 + 8 \cdot 64} = \frac{80}{544} = \frac{5}{34}$.

$\mathbf{x}_2 = (2, 1)^T - \frac{5}{34}(4, 8)^T = (2 - \frac{20}{34},\ 1 - \frac{40}{34})^T = (\frac{24}{17}, -\frac{3}{17})^T$.

(2) $\mathbf{s}_1 = \mathbf{x}_2 - \mathbf{x}_1 = (-\frac{10}{17}, -\frac{20}{17})^T$. $\quad \nabla f(\mathbf{x}_2) = (\frac{48}{17}, -\frac{24}{17})^T$.

$\mathbf{y}_1 = \nabla f(\mathbf{x}_2) - \nabla f(\mathbf{x}_1) = (\frac{48}{17} - 4,\ -\frac{24}{17} - 8)^T = (-\frac{20}{17}, -\frac{160}{17})^T$.

$\mathbf{y}_1^T \mathbf{s}_1 = \frac{200}{289} + \frac{3200}{289} = \frac{3400}{289} > 0$. ✓

(3)–(4) The BFGS update gives B_2 closer to $\text{diag}(2, 8)$ than $B_1 = I$—the approximation improves along the step direction. (Full computation in the homework.)

Exercise 2: Inverse Update by Hand

Exercise 2

Given $H_1 = I_2$, $\mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, $\mathbf{y}_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$.

1. Compute ρ_1, V_1, W_1, and H_2 using the BFGS inverse update formula.
2. Verify $H_2\mathbf{y}_1 = \mathbf{s}_1$.
3. Compute B_2 via the forward update and verify $B_2^{-1} = H_2$.
4. Find the eigenvalues of H_2. Is it positive definite?

(This is the example from the homework—try it on your own first!)

Solution: Exercise 2

Solution

(1) $\rho_1 = \frac{1}{\mathbf{y}_1^T \mathbf{s}_1} = \frac{1}{2}$.

$$V_1 = I - \tfrac{1}{2}\begin{pmatrix}2\\1\end{pmatrix}(1,0) = \begin{pmatrix}0 & 0\\ -\frac{1}{2} & 1\end{pmatrix},$$

$$W_1 = I - \tfrac{1}{2}(1,0)^T \begin{pmatrix}2 & 1\end{pmatrix} = \begin{pmatrix}0 & -\frac{1}{2}\\ 0 & 1\end{pmatrix}.$$

$$H_2 = W_1 V_1 + \tfrac{1}{2}\begin{pmatrix}1 & 0\\ 0 & 0\end{pmatrix} = \begin{pmatrix}\frac{1}{4} & -\frac{1}{2}\\ -\frac{1}{2} & 1\end{pmatrix} + \begin{pmatrix}\frac{1}{2} & 0\\ 0 & 0\end{pmatrix} = \begin{pmatrix}\frac{3}{4} & -\frac{1}{2}\\ -\frac{1}{2} & 1\end{pmatrix}.$$

(2) $H_2\mathbf{y}_1 = \begin{pmatrix}\frac{3}{2} - \frac{1}{2}\\ -1+1\end{pmatrix} = \begin{pmatrix}1\\0\end{pmatrix} = \mathbf{s}_1$. ✓

(3) $B_2 = \begin{pmatrix}2 & 1\\ 1 & \frac{3}{2}\end{pmatrix}$, $\det(B_2) = 2$, $B_2^{-1} = \begin{pmatrix}\frac{3}{4} & -\frac{1}{2}\\ -\frac{1}{2} & 1\end{pmatrix} = H_2$. ✓

(4) $\lambda = \frac{7 \pm \sqrt{17}}{8}$, i.e., $\lambda \approx 1.39$ and $\lambda \approx 0.36$. Both positive $\Rightarrow$ PD.

Exercise 3: Why the Curvature Condition Matters

Exercise 3

Suppose $B_k = I_2$ and we observe:

$$\mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \mathbf{y}_1 = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$$

1. Compute $\mathbf{y}_1^T \mathbf{s}_1$. Is the curvature condition satisfied?
2. Apply the BFGS formula anyway. What happens? Compute B_2 and check if it is positive definite.
3. What does this tell you about the importance of the Wolfe line search?

Solution: Exercise 3

Solution

(1) $\mathbf{y}_1^T \mathbf{s}_1 = -1 + 0 = -1 < 0$. The curvature condition is violated.

(2) Applying the BFGS formula:

$$\frac{B_k \mathbf{s}_1 \mathbf{s}_1^T B_k}{\mathbf{s}_1^T B_k \mathbf{s}_1} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad \frac{\mathbf{y}_1 \mathbf{y}_1^T}{\mathbf{y}_1^T \mathbf{s}_1} = \frac{1}{-1} \begin{pmatrix} 1 & -2 \\ -2 & 4 \end{pmatrix} = \begin{pmatrix} -1 & 2 \\ 2 & -4 \end{pmatrix}$$

$$B_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} -1 & 2 \\ 2 & -4 \end{pmatrix} = \begin{pmatrix} -1 & 2 \\ 2 & -3 \end{pmatrix}$$

Eigenvalues: $\lambda = -2 \pm \sqrt{8}$, so $\lambda_1 \approx 0.83$, $\lambda_2 \approx -4.83$. Not PD!

(3) Without the curvature condition, BFGS can produce an indefinite B_{k+1}, meaning the search direction may not be a descent direction. The Wolfe line search guarantees $\mathbf{y}_k^T \mathbf{s}_k > 0$, preventing this failure.

Key Takeaways

The BFGS Method

- Approximates the Hessian using only gradient information via the secant condition
- Rank-2 update: forgets old curvature in step direction, learns new curvature from gradient differences
- Preserves positive definiteness when $\mathbf{y}_k^T \mathbf{s}_k > 0$ (guaranteed by Wolfe conditions)

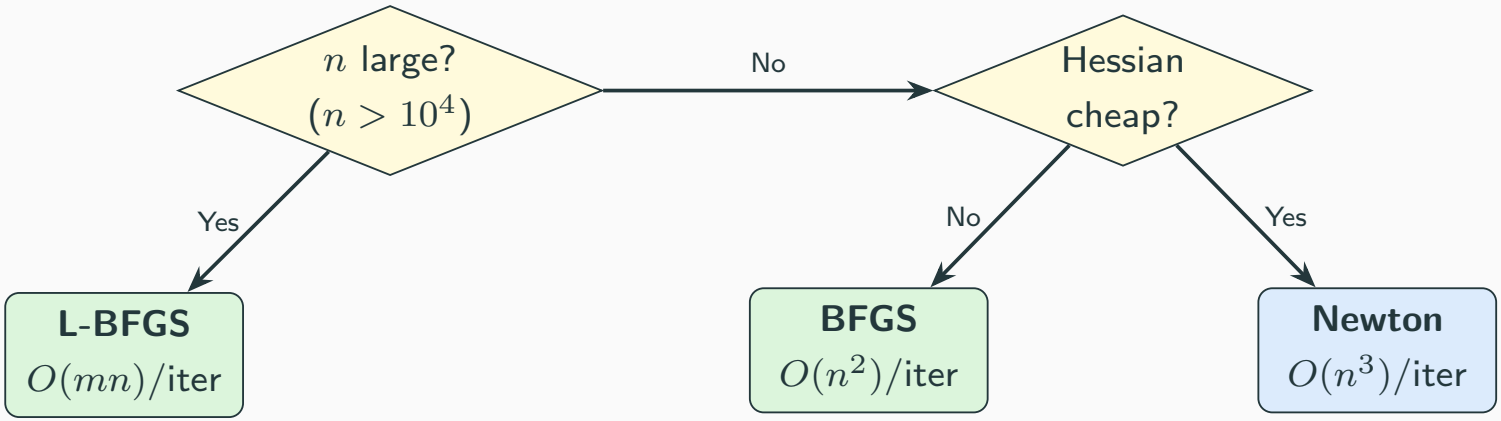
Two Implementation Strategies

- **Forward update** (B_k): requires solving $B_k \mathbf{d}_k = -\nabla f(\mathbf{x}_k)$ at $O(n^3)$
- **Inverse update** ($H_k = B_k^{-1}$): direction is $\mathbf{d}_k = -H_k \nabla f(\mathbf{x}_k)$ at $O(n^2)$
- Sherman–Morrison formula converts one to the other

Convergence and Scalability

- Superlinear convergence: nearly as fast as Newton, much cheaper per iteration
- L-BFGS: uses $O(mn)$ memory for large-scale problems ($m \approx 5$–20)
- Most popular quasi-Newton method in practice

Method Comparison: A Decision Guide



Method	Cost/iter	Memory	Convergence
Steepest Descent	$O(n)$	$O(n)$	Linear
L-BFGS	$O(mn)$	$O(mn)$	Superlinear
BFGS	$O(n^2)$	$O(n^2)$	Superlinear
Newton	$O(n^3)$	$O(n^2)$	Quadratic

Looking Ahead

Next Time: Conjugate Gradient Methods

- Conjugate directions: an elegant concept for quadratics
- Conjugate gradient (CG): finite convergence in at most n steps
- CG as a bridge between steepest descent and Newton
- Extensions to nonlinear optimization (Fletcher–Reeves, Polak–Ribière)
- Connection to Krylov subspace methods

Reading:

- Nocedal & Wright: Chapter 6 (Quasi-Newton Methods), Chapter 7 (L-BFGS)
- Bazaraa, Sherali, Shetty: Sections 8.6–8.8
- Course notes: Sections 14.8–14.9

Questions?

ISE 5406 — Optimization II

Virginia Tech, Spring 2026